A Search Engine for Video-Game Knowledge

Text analysis, indexing and Boolean retrieval with two competing pipelines

Priyanka B · Sachin B · Tiya P · Vaidhav A

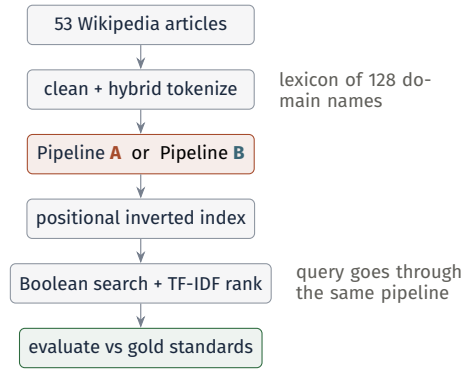
Course code DATA403: Natural Language Processing · Assessment-1
Vidyashilp University, Bangalore

October 2026

What we built

A complete classical search engine, measured at every stage

- Indexer (ahead of time): reads every document, builds a lookup table of every term and where it occurs
- Searcher (at question time): looks terms up, combines them with `AND` / `OR` / `NOT`, ranks the result
- Everything in between decides which "words" exist at all
- Two pipelines make opposite bets on that decision; we compare them under rules fixed in advance



Domain and data

Why video games, and where the documents come from

Hard for generic NLP tools

- hyphens and numbers in names: *Counter-Strike*, *Destiny 2*
- names made of everyday words: *Tomb Raider*
- spelling variants: *Game Boy* / *Gameboy*, *SNES*

Corpus from MIRACL (Zhang et al. 2023)

- 12 real questions; their judged articles: relevant + hard negatives
- full Wikipedia articles, **D01–D53**

Documents	53
Sentences	9,305
Tokens (hybrid)	251,482
Characters	1,362,150
Vocabulary (case-folded)	16,030
Avg. length (tokens)	4,744.9

Corpus statistics (`corpus_summary.csv`)

53 > 15–30 recommended: each question needs several relevant documents and some hard negatives, or precision and recall are meaningless.

Gold standards: an answer key for every stage

Gold standard	Contents	Evaluates
Relevance judgments	12 questions × 53 documents = 636 labels (57 relevant)	searcher, ranker
POS gold	226 tokens, 8 corpus sentences, Penn Treebank	5 POS taggers
NER gold	51 entities, 7 types (PERSON … EVENT)	2 entity recognisers
Domain lexicon	128 names that must survive as one term	tokenizers, pipelines
Stemming groups	word families + a trap: *universe* vs *university*	stemmers, lemmatizers

Process discipline

Hypotheses, metrics and the selection rule were preregistered before any result; every later change is a dated amendment in the decision log. Relevance labels: two reviewers under a written protocol, so search results are labelled *preliminary*.

Select → Order → Implement → Compare → Evaluate → Justify

Choose each component by measuring it on the corpus

Tokenization: where does a word end?

The single most important choice

Tokenizer	Tokens	Term dict.	Names kept
NLTK	253,416	16,619	0 / 91
spaCy	261,849	15,745	0 / 91
Custom (regex)	252,750	16,664	0 / 91
Hybrid	251,482	16,030	**91 / 91**

`tokenization_comparison.csv`, `tokenizer_domain_terms.csv`

`Counter-Strike: Global Offensive`
spaCy: `Counter | - | Strike | : | Global | Offensive`
hybrid: `Counter-Strike_Global_Offensive`

Hybrid = custom regex rules + three domain steps:

1. rewrite variants (*Gameboy* → *Game Boy*)
2. split lower-case compounds (*first-person*)
3. merge lexicon names, longest first (`Game_Boy`)

Custom rules keep *$8.5*, *U.S.*, *PS4*, *1990s* whole.

Chosen for both pipelines. Limit: only names in the lexicon are protected.

Subword tokenization (BPE)

Useful for vocabulary control, wrong for an index of names

Method	Tokens	Vocabulary
Hybrid	251,482	18,192
GPT-2 BPE (tiktoken)	284,579	18,127
Corpus BPE (1,000 merges)	423,824	3,242

`bpe_vs_tokenizers.csv`

Word	GPT-2	Corpus BPE
nintendo	n | intendo	nintendo</w>
metroid	met | roid	metroid</w>
multiplayer	multi | player	multi | player</w>

`bpe_results.csv`

- BPE (Sennrich et al. 2016) never meets an unknown word and keeps the vocabulary tiny
- But pieces like *met | roid* are not searchable units: users type words and names
- Frequent domain words are learned whole by the corpus model, not by GPT-2 → analysed, not used for the index

Stop words, stemming, lemmatization

Scored with Paice's under- and over-stemming indices

Method	UI ↓	OI ↓	univers{e,ity}?
Porter	0.298	0.0087	merged
Snowball	0.298	0.0087	merged
Lancaster	0.000	0.0087	merged
WordNet (verb)	0.617	0	kept apart
spaCy lemma	0.574	0	kept apart

`stemming_lemmatization.csv` (Paice 1994)

- every stemmer over-stems the trap
- lemmatizers never over-merge but need the POS tag (*saw*: noun or verb?)
- **Order matters**: Porter *before* stop-word removal leaks `thi`, `wa`, `ha` into the index
- stop lists: 10 words removes 22.3% of tokens, NLTK's 198 removes 38.1%

Choices: **B** uses Snowball (no `thi`/`wa` leaks); **A** uses spaCy lemmas in context, never touching proper nouns, numbers or merged names.

POS tagging and named entities

General models do not know the domain; small domain knowledge fixes it

POS tagger	Accuracy
NLTK (default)	0.982
spaCy (default)	0.951
Rules + dictionary (custom)	0.987
KNN (custom, ML)	0.602

226 gold tokens; `pos_tagging_results.csv`

NER (51 entities)	Correct	Wrong	Missed
spaCy default	29	13	9
+ EntityRuler	37	6	8

`ner_summary.csv`; mean F1 0.563 → 0.744

- spaCy calls *Prey*, *Doom*, *Halo* PERSON; the lexicon-driven EntityRuler relabels them PRODUCT
- The dictionary tagger improves NLTK by one token: with 226 gold tokens, a gain within noise; KNN is far behind

N-grams: which word sequences recur?

Counted over each pipeline's index terms, $n = 1 \ldots 5$

	distinct		meaningful (top 20)	
n	**A**	**B**	**A**	**B**
1	13,165	11,247	–	–
2	108,872	98,356	0.30	1.00
3	159,589	128,736	0.55	1.00
4	171,416	134,305	0.65	1.00
5	173,966	135,602	0.85	1.00

`ngram_summary.csv`

- the dictionary explodes with *n*: about 99% of 5-grams are distinct, so a phrase needs count ≥ 2
- **A** keeps function words: top bigrams *it be*, *be release*
- **B** removed them: *lara croft*, *smash bros*
- top bigram in both: *video game* (**A** 516, **B** 536)

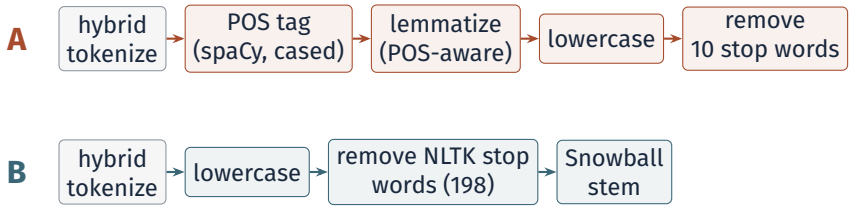
Meaningful = occurs ≥ 2, no stop word or bare number at either edge, contains a letter. Frequent unmerged phrases are candidates for the lexicon.

Select → **Order** → Implement → Compare → Evaluate → Justify

Combine the same components in two orders that make opposite bets

The two pipelines

Same ingredients, different order



A: annotate first (linguist)

Tag while capitals and context still exist, then reduce to dictionary forms.
Bet: careful analysis keeps names and meanings intact.

B: normalise first (IR engineer)

Lowercase at once, drop all common words, chop to stems (Manning et al. 2008).
Bet: aggressive merging, more matches, smaller index.

Why each step sits where it does

Every process: dependency, and what breaks if it moves

Process	Position	Depends on	If moved
Cleaning	first	raw text	artefacts (`[12]`, *é*) become tokens
Name merging	inside tokenization	original case, stop words present	after stop removal or stemming, names stop matching the lexicon
POS tagging (**A**)	before lowercasing	case, context	*Halo* loses its proper-noun clue and gets lemmatized
Lemmatization (**A**)	after tagging	POS tags	without POS, WordNet under-merges (UI 0.62–0.87)
Stop words (**B**)	before stemming	lowercase	stem-first leaks `thi`, `wa`, `ha`
Query processing	same as documents	the pipeline	query terms would not match index terms

One sentence through both pipelines

"The Legend of Zelda: Majora's Mask was released for the Nintendo 64 in 2000 and players were praising it."

Token	POS	**A** term	**B** term
The_Legend_of_Zelda	NN	the_legend_of_zelda	the_legend_of_zelda
Nintendo_64	NNP	nintendo_64	nintendo_64
was	VBD	be	(removed)
released	VBN	release	releas
players	NNS	player	player
praising	VBG	praise	prais

`pipeline_trace_example.csv` (abridged)

- **A** keeps real words; **B** keeps fewer, truncated stems
- both protect names identically: the tokenizer merged them before either pipeline started

Select → Order → **Implement** → Compare → Evaluate → Justify

Build the indexer and the Boolean searcher, shared by both pipelines

The positional inverted index

The data structure at the heart of the system

term → document → positions

```
"game_boy": {"D01": [3893], "D03": [1978],
             "D05": [5237], ...}   12 docs
"metroid":  {"D01": [2160], "D05": [5808, 7090],
             "D08": [790, 1091, 1558], ...}
```

- positions make phrase search possible
- same idea as Lucene / Elasticsearch, without compression

	A	B
Terms	13,165	11,247
Postings	53,258	47,557
Positions	176,387	137,253
In only 1 doc	54%	53%

index_structure.csv

Zipf: most terms are rare and carry the information; terms in every document get an idf of 0.

The searcher and the 12 test queries

Boolean decides *which* documents; ranking decides *in what order*

1. query runs through the same pipeline as the documents
2. AND = intersect, OR = union, NOT = subtract posting lists; phrases checked by position
3. matches ranked by TF-IDF cosine, title matches $\times 2$

	Query	Type
Q01	`Total War`	keyword (bigram)
Q03	`Nintendo AND PlayStation`	AND
Q04	`"Ash Ketchum" AND NOT Japanese`	NOT
Q06	`Zelda AND (theme OR song)`	OR
Q07	`"Tomb Raider"`	phrase
Q09	`"Guitar Hero Live"`	phrase (trigram)
Q12	`SNES OR "Super Nintendo …"`	OR

7 of the 12 queries; every type covered

Select → Order → Implement → **Compare** → **Evaluate** → Justify

Predictions first, then the numbers

What we predicted, before running anything

Six preregistered hypotheses

	Prediction	Evidence	Verdict
H1	**B** has fewer tokens and a smaller vocabulary	176,387 vs 137,253; 13,165 vs 11,247	**supported**
H2	**B** has higher recall	R 0.796 vs 0.796	no difference
H3	**A** has higher precision, esp. phrases	P 0.596 vs 0.596	no difference
H4	**B** makes false phrase matches	16 vs 28 docs (probe)	**supported***
H5	**A** preserves $\geq$5 pts more names	1.000 vs 0.898	**supported**
H6	**A** has more meaningful bigrams	0.30 vs 1.00	**rejected**

`hypotheses.csv`. *exploratory: not testable on the 12 queries

Reporting the predictions that failed is part of the result: H2, H3 and H6 did not hold.

Pipeline comparison

Table H: the brief's required comparison

Measure	A	B	Final (A)
Token count	176,387	137,253	176,387
Vocabulary size	13,165	11,247	13,165
Meaningful bigrams	0.30	1.00	0.30
Name preservation	1.000	0.898	1.000
Precision (macro)	0.596	0.596	0.596
Recall (macro)	0.805	0.805	0.805
F1 (macro)	0.648	0.648	0.648
P@5 / P@10	.583 / .367	.583 / .367	.583 / .367

`pipeline_comparison.csv`

- **B** index 29% smaller
- **A** keeps all 128 names; **B** breaks 13 (*Civilization* → `civil`)
- **identical search results** on all 12 queries
- the bigram test favours **B** by construction (it checks the stop list **B** removed)

Retrieval performance

Why accuracy is the wrong metric for search

Query type	P	R	F1
AND	0.722	0.963	0.798
phrase	0.738	0.833	0.778
keyword	0.636	0.875	0.643
OR	0.400	0.929	0.556
NOT	0.350	0.333	0.325
All 12	0.596	0.805	0.648

`retrieval_by_query_type.csv` (same for **A** and **B**)

	relevant	not relevant
retrieved	TP 48	FP 56
not retrieved	FN 9	TN 523

Confusion matrix, all 636 pairs

Accuracy misleads

Our searcher: accuracy **0.898**. Returning *nothing*: **0.910**. Most documents are irrelevant to any query, so we report precision and recall.

Where the searcher goes wrong

Every error classified: 9 misses, 56 false hits, same for both pipelines

Cause	FN	FP
Passing mention	0	54
Query formulation	6	0
Wrong relevance label	3	2
Any preprocessing step	0	0

`error_analysis.csv`

- **NOT throws away answers**: `"Ash Ketchum" AND NOT Japanese` loses the *Ash Ketchum* article, which mentions his Japanese voice actress
- **Passing mentions**: `Metroid` returns 11 documents; the 3 relevant ones rank 1–3, the rest are console histories

No error comes from tokenization, stemming, lemmas, stop words or NER, which is why **A** and **B** tie.

Select → Order → Implement → Compare → Evaluate → **Justify**

Apply the rule fixed in advance, and explain what the numbers mean

Choosing the final pipeline

A rule fixed in advance, applied in order

1. **F1 gap ≥ 0.05?** **A** 0.648, **B** 0.648, gap 0.000 $\rightarrow$ does not decide
2. **Domain-term preservation gap ≥ 0.05?** **A** 1.000, **B** 0.898, gap 0.102 $\rightarrow$ **decides**
3. Meaningful-bigram rate: not reached

Final pipeline: A (annotate first, lemmas)

A is *not* chosen because it searches better; on this corpus it does not. It is chosen because, when both find the same documents, **A**'s index represents the domain's names faithfully: the term dictionary, postings and n-grams a person reads are real words and intact names.

The cost of **A**: a 29% larger index and slower indexing (spaCy tagger). Query time: a few ms for both, within run-to-run noise.

An earlier run chose B: what we learned

One oddly worded query decided a whole comparison

- first run: queries worded automatically from each question's first content words
- *"Was there a Sonic game on the Sega Genesis?"* became "`there Sonic game`"
- **B** drops *there* as a stop word and finds the answers; **A** keeps it and finds nothing

F1 (automatic wording)	A	B
all 12 queries	0.183	0.237
without Q02	0.200	0.176

reproduced exactly, 12/12 queries

What we did about it

Queries rewritten by hand under a stated rule, *before* any new result existed; the amendment is dated and both runs are reported. **Finding:** differences between the pipelines come from stop words and inflections in the *query wording*, not from how they handle domain content.

Why the pipelines agree, and the design lesson

Why they tie

- both share the component that matters most: the name-merging tokenizer
- our queries are about names; `tomb_raider` is never stemmed or lemmatized
- **A** and **B** differ on verb endings and function words, which the queries never touch

Design lesson

On an entity-heavy domain, the decisive preprocessing choice is the **domain-aware tokenizer and lexicon**, not stemming vs lemmatization.

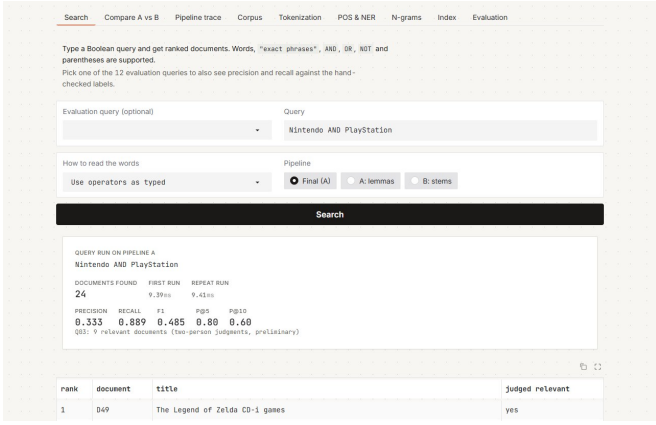
Consistent with Brin and Page (1998), who launched without stemming. Exploratory: no pipeline was run without name merging.

The GUI: a working searcher with an X-ray

9 tabs covering all 15 required functions

- **Search**: Boolean query, type, pipeline; ranked results, time, P / R / F1
- **Compare A vs B**: side by side, with a diff
- **Pipeline trace**: any sentence, stage by stage, timed
- **Corpus**, **Tokenization**, **POS & NER**, **N-grams**
- **Index**: postings with positions and context
- **Evaluation**: the decision and every number

Every button calls the same pipelines and index as the report. Right: Search on Q03 with timing, precision, recall and the judged-relevant column.



Limitations and next steps

Limitations

- relevance labels: 2 reviewers, no agreement measure, ≥ 5 labels look wrong (Voorhees 2000)
- 12 queries: too few for significance tests
- corpus built around its queries: absolute scores optimistic, A vs B unaffected
- lexicon of 128 names: unknown games still split

Next steps

- blind relevance judging, more annotators
- 72 eligible questions; wordy queries to separate H2–H4
- one-component-at-a-time ablation with a paired test
- grow the lexicon from frequent n-grams (*smash bros*); BM25 ranking
- a small knowledge graph from NER output

Conclusion

1. **The tokenizer matters most**: only a domain-aware tokenizer keeps names intact (91/91 vs 0/91).
2. **Order is visible**: stem before stop-word removal and `thi`, `wa`, `ha` leak; tag before lowercasing or proper nouns get lemmatized.
3. **General models miss the domain**: an EntityRuler lifts NER F1 from 0.563 to 0.744.
4. **Both pipelines search identically** (P 0.596, R 0.805, F1 0.648); the preregistered rule selects Pipeline A on name preservation.
5. **Accuracy misleads**: 0.898 for our searcher, 0.910 for returning nothing.

All search results are preliminary until the relevance labels are re-judged by more annotators.

Bibliography I

- Brin, Sergey and Lawrence Page (1998). “The Anatomy of a Large-Scale Hypertextual Web Search Engine”. In: *Proceedings of the Seventh International World Wide Web Conference (WWW7)*. URL: http://infolab.stanford.edu/~backrub/google.html.
- Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze (2008). *Introduction to Information Retrieval*. Cambridge University Press.
- Paice, Chris D. (1990). “Another Stemmer”. In: *ACM SIGIR Forum* 24.3, pp. 56–61.
- — (1994). “An Evaluation Method for Stemming Algorithms”. In: *Proceedings of SIGIR ’94*, pp. 42–50. DOI: 10.1007/978-1-4471-2099-5_5.
- Porter, Martin F. (1980). “An Algorithm for Suffix Stripping”. In: *Program* 14.3, pp. 130–137.
- Sennrich, Rico, Barry Haddow, and Alexandra Birch (2016). “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Bibliography II

- Voorhees, Ellen M. (2000). “Variations in Relevance Judgments and the Measurement of Retrieval Effectiveness”. In: *Information Processing & Management* 36.5, pp. 697–716.
- Zhang, Xinyu et al. (2023). “MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages”. In: *Transactions of the Association for Computational Linguistics*.

Thank You for Your Attention!

Questions? The GUI is ready for a live demo.